Universidade de Coimbra
Faculdade de Ciências e Tecnologia
Departamento de Engenharia Informática

Segurança em Tecnologias da Informação

Project 2 – IPTables and Snort

Teachers

António Jorge da Costa Granjal

Team

Carlos João Lopes Mendes, nº2017257272

Filipe Miguel Fonseca dos Santos, nº2017271196

Index

Configurations 3
Network Configurations 3
FTP – Necessary Preparations 4
Stopping Ports 22 and 53 so we can Run Netcat 4
Packet filtering and NAT using IPTables 5
Firewall configuration to protect the router 5
Firewall configuration to authorize direct communications (without NAT) 6
Firewall configuration for connections to the external IP address of the firewall (using NAT) 9
Firewall configuration for communications from the internal network to the outside (using NAT) 10
Snort 12
IPTables rules related with snort 12
SQL Injection Attacks, XSS Attacks and Snort Rules to prevent them 12
Tests 13
IPTables Testing 13
Snort Testing 14
Important Annexes 15
Other Annexes 15
References 15

Configurations

Network Configurations

The networks configurations follow the addresses and networks shown in figure 1. In general, we followed the example configurations, only changing the addresses for the dns2 and eden, so they match the network used by the respective interface (87.248.214.0/24). The numbers we assigned for the services on each network are all presented in this figure.

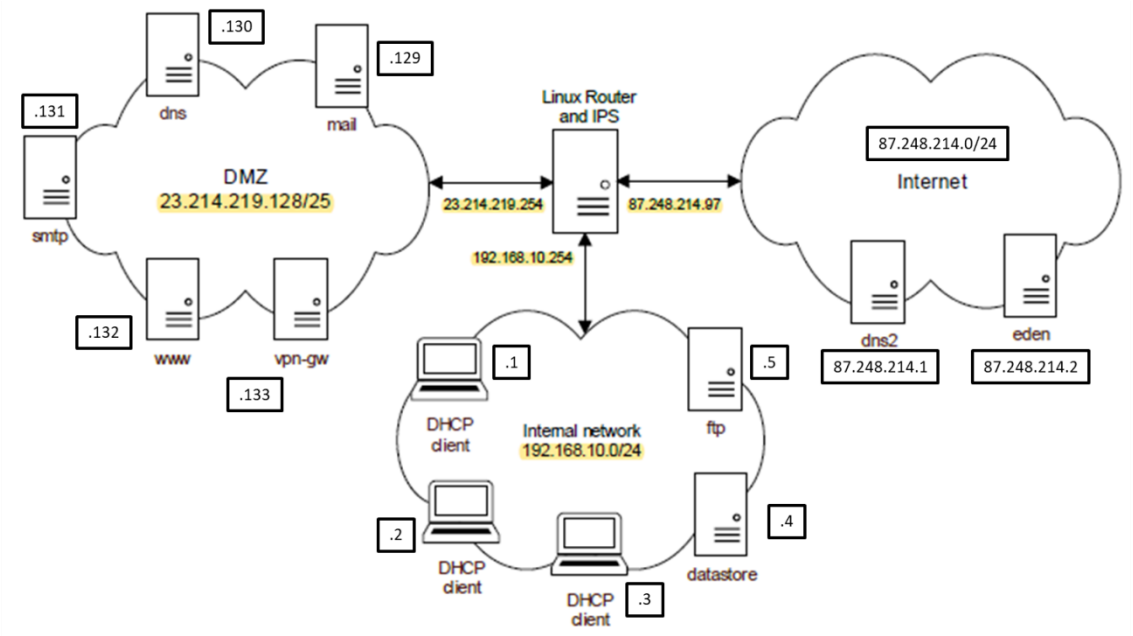

Figure 1 – Network Configuration

We used 3 virtual machines for the assignment. Machine 1 is always the router. Machine 2 can be used for either the DMZ or the Internal Network, while machine 3 can be used for either the Internet or the Internal Network. We change the interface being used for machines 2 and 3 according to the test being performed.

On top of that, we make sure to add the necessary routes whenever we change the network on an interface. The routes are added following the logic from figure 2. This is done to guarantee connectivity between networks.

```
Máquina 2:
    (DMZ to Internet)
        route add -net 87.248.214.0/24 gw 23.214.219.254
    (Internal Network to Internet)
        route add -net 87.248.214.0/24 gw 192.168.10.254
    (DMZ to Internal Network)
        route add -net 192.168.10.0/24 gw 23.214.219.254

Máquina 3:
    (Internet to DMZ)
        route add -net 23.214.219.128/25 gw 87.248.214.97
    (Internet to Internal Network)
        route add -net 192.168.10.0/24 gw 87.248.214.97
    (Internal Network to DMZ)
        route add -net 23.214.219.128/25 gw 192.168.10.254
```

Figure 2 – Routes Added

FTP – Necessary Preparations

In order to test the FTP ports correctly, it's necessary to use active and passive FTP, instead of the usual netcat tests (which would be incomplete). As such, we need to be able to run FTP servers on machines 2 and 3, which requires the following configuration.

```
ftp (Instalar o ftp)
yum install vsftpd
systemctl start vsftpd
systemctl enable vsftpd

gedit /etc/vsftpd.conf
    local_enable=YES
    write_enable=YES
    pasv_enable=YES
    pasv_promiscuous=YES
    -> Fazer restart do vsftpd (systemctl restart vsftpd)
```

Figure 3 – FTP Installation

We also need to enable tracking for ftp connections, in order to deal with the issue of dynamic ports in ftp's passive mode, an issue that will be further explained when we get to the ftp related rules. This was done using **[3]** as a guide.

```
Enable nf_conntrack_ftp and other necessary modules for ftp
    gedit /etc/sysconfig/iptables-config
        -> IPTABLES_MODULES="nf_conntrack_ftp nf_conntrack_tftp nf_nat_ftp nf_nat_tftp"
    echo 1 > /proc/sys/net/netfilter/nf_conntrack_helper
    gedit /etc/modprobe.d/iptables.conf
        -> options nf_conntrack_ftp ports=21
        -> options nf_nat_ftp ports=21
    systemctl restart iptables

    lsmod | grep ftp
```

Figure 4 – Enable tracking for ftp connections

Stopping Ports 22 and 53 so we can Run Netcat

These ports are usually already being used, so it's necessary to stop them, in order to run netcat on them.

For port 22 we can simply do "systemctl stop sshd".

As for port 53, we used the method shown in figure 5, using the explanation in **[1]** as a starting point.

```
gedit /etc/systemd/resolved.conf
    -> Add/Uncomment
        DNS=1.1.1.1
        DNSStubListener=no
sudo ln -sf /run/systemd/resolve/resolv.conf /etc/resolv.conf
reboot
```

Figure 5 – Stop port 53

Packet filtering and NAT using IPTables

Firewall configuration to protect the router

```
"The firewall configuration should drop all communications entering the router system"

    (Policy):
        iptables -P OUTPUT DROP
        iptables -P INPUT DROP
```

Figure 6 – Input and Output Drop Policies

To drop all communications entering the router we simply change the INPUT policy to DROP. Although it wasn't requested, we decided it would make sense to also do the same for the OUTPUT, as this is in line with the other policies used in the assignment.

```
"DNS name resolution requests sent to outside servers."

    domain          53/tcp                          # name-domain server
    (From router to dns)
        iptables -A INPUT -s 23.214.219.130 -p tcp --sport domain -j ACCEPT
        iptables -A OUTPUT -d 23.214.219.130 -p tcp --dport domain -j ACCEPT
    (From router to dns2)
        iptables -A INPUT -s 87.248.214.1 -p tcp --sport domain -j ACCEPT
        iptables -A OUTPUT -d 87.248.214.1 -p tcp --dport domain -j ACCEPT
    (From router to eden)
        iptables -A INPUT -s 87.248.214.2 -p tcp --sport domain -j ACCEPT
        iptables -A OUTPUT -d 87.248.214.2 -p tcp --dport domain -j ACCEPT

    domain          53/udp
    (From router to dns)
        iptables -A INPUT -s 23.214.219.130 -p udp --sport domain -j ACCEPT
        iptables -A OUTPUT -d 23.214.219.130 -p udp --dport domain -j ACCEPT
    (From router to dns2)
        iptables -A INPUT -s 87.248.214.1 -p udp --sport domain -j ACCEPT
        iptables -A OUTPUT -d 87.248.214.1 -p udp --dport domain -j ACCEPT
    (From router to eden)
        iptables -A INPUT -s 87.248.214.2 -p udp --sport domain -j ACCEPT
        iptables -A OUTPUT -d 87.248.214.2 -p udp --dport domain -j ACCEPT
```

Figure 7 – Accept rules for the DNS Input and Output

In this case, we considered DNS requests sent to the 3 servers in the network scenario (dns, dns2 and eden, in this order). We consider the destination port in the OUTPUT, since we are sending the requests from the router to those servers. The INPUT rule is necessary so we can receive replies. We also do the same for the udp, as DNS uses both tcp (for communications between DNS servers) and udp (for name resolutions).

It's worth noting that we could have also done a more general policy, without defining any specific addresses, but we decided to be more specific given the scenario.

```
"SSH connections to the router system, if originated at the internal network or at the VPN gateway (vpn-gw)."

    ssh             22/tcp                          # The Secure Shell (SSH) Protocol
        iptables -A INPUT -s 192.168.10.0/24 -p tcp --dport ssh -j ACCEPT
        iptables -A OUTPUT -d 192.168.10.0/24 -p tcp --sport ssh -j ACCEPT

        iptables -A INPUT -s 23.214.219.133 -p tcp --dport ssh -j ACCEPT
        iptables -A OUTPUT -d 23.214.219.133 -p tcp --sport ssh -j ACCEPT
```

Figure 8 - Accept rules for the SSH Input and Output

Here we do something similar for the SSH. In this case however, the destination port is used in the INPUT. This is done since SSH connections come from the internal network or the VPN gateway to the router, unlike the previous example (where the connections went from the router to the outside).

Firewall configuration to authorize direct communications (without NAT)

```
"The firewall configuration should drop all communications between networks"

    (Policy):
        iptables -P FORWARD DROP
```

Figure 9 – Forward Drop Policy

For this one we put the Forward policy as drop, so all communications between networks are dropped.

```
"Domain name resolutions using the dns server."

    domain          53/tcp                          # name-domain server
        iptables -A FORWARD -d 23.214.219.130 -p tcp --dport domain -j ACCEPT
        iptables -A FORWARD -s 23.214.219.130 -p tcp --sport domain -j ACCEPT


    domain          53/udp
        iptables -A FORWARD -d 23.214.219.130 -p udp --dport domain -j ACCEPT
        iptables -A FORWARD -s 23.214.219.130 -p udp --sport domain -j ACCEPT
```

Figure 10 - Accept rules for the SSH

Since we are requesting domain name resolutions from the dns server, the destination port “domain” is on the rule that has the dns server as a destination. The complementary rules are used so that there is a reply (in this case we use source and source port).

```
"The dns server should be able to resolve names using DNS servers on the Internet (dns2 and also others)."


    domain          53/tcp                          # name-domain server
    (from dns to dns2)
        iptables -A FORWARD -d 87.248.214.1 -s 23.214.219.130 -p tcp --dport domain -j ACCEPT
        iptables -A FORWARD -s 87.248.214.1 -d 23.214.219.130 -p tcp --sport domain -j ACCEPT
    (from dns to eden)
        iptables -A FORWARD -d 87.248.214.2 -s 23.214.219.130 -p tcp --dport domain -j ACCEPT
        iptables -A FORWARD -s 87.248.214.2 -d 23.214.219.130 -p tcp --sport domain -j ACCEPT


    domain          53/udp
    (from dns to dns2)
        iptables -A FORWARD -d 87.248.214.1 -s 23.214.219.130 -p udp --dport domain -j ACCEPT
        iptables -A FORWARD -s 87.248.214.1 -d 23.214.219.130 -p udp --sport domain -j ACCEPT
    (from dns to eden)
        iptables -A FORWARD -d 87.248.214.2 -s 23.214.219.130 -p udp --dport domain -j ACCEPT
        iptables -A FORWARD -s 87.248.214.2 -d 23.214.219.130 -p udp --sport domain -j ACCEPT
```

Figure 11 - Accept rules from the DNS to the Internet

Here we need to define both a destination and a source, since we want to go specifically from the dns server to the dns servers on the internet. The destination port is placed on the same rules that have the internet addresses as destination, since those are considered the servers in this case.

```
"The dns and dns2 servers should be able to synchronize the contents of DNS zones."

    domain          53/tcp                          # name-domain server
    (from dns2 to dns)
        iptables -A FORWARD -d 23.214.219.130 -s 87.248.214.1 -p tcp --dport domain -j ACCEPT
        iptables -A FORWARD -s 23.214.219.130 -d 87.248.214.1 -p tcp --sport domain -j ACCEPT

    domain          53/udp
    (from dns2 to dns)
        iptables -A FORWARD -d 23.214.219.130 -s 87.248.214.1 -p udp --dport domain -j ACCEPT
        iptables -A FORWARD -s 23.214.219.130 -d 87.248.214.1 -p udp --sport domain -j ACCEPT
```

Figure 12 - Accept rules from the dns2 to the dns server

Thanks to the rules implemented in the previous point, the dns server can already reach out to the dns2. As such, we only need to make the opposite happen, so that the dns2 can use the dns server as a destination. For this to happen, we use the destination ports in the rules that have the dns server as a destination, as that is the server for this situation. With these rules and the previous ones in place, the dns and dns2 servers can now synchronize with each other.

```
"SMTP connections to the smtp server."

    smtp            25/tcp          mail
        iptables -A FORWARD -d 23.214.219.131 -p tcp --dport smtp -j ACCEPT
        iptables -A FORWARD -s 23.214.219.131 -p tcp --sport smtp -j ACCEPT
```

Figure 13 - Accept rules for SMTP connections

Since no source is specified, we only need to write the address of the smtp server. The destination port is together with the destination, as that is where the server lies.

```
"POP and IMAP connections to the mail server."

    pop2            109/tcp         pop-2 postoffice        # POP version 2
        iptables -A FORWARD -d 23.214.219.129 -p tcp --dport pop2 -j ACCEPT
        iptables -A FORWARD -s 23.214.219.129 -p tcp --sport pop2 -j ACCEPT


    pop3            110/tcp         pop-3           # POP version 3
        iptables -A FORWARD -d 23.214.219.129 -p tcp --dport pop3 -j ACCEPT
        iptables -A FORWARD -s 23.214.219.129 -p tcp --sport pop3 -j ACCEPT


    imap            143/tcp         imap2           # Interim Mail Access Proto v2
        iptables -A FORWARD -d 23.214.219.129 -p tcp --dport imap -j ACCEPT
        iptables -A FORWARD -s 23.214.219.129 -p tcp --sport imap -j ACCEPT
```

Figure 14 - Accept rules for pop2, pop3 and imap connections

These follow the same logic as the one used for the smtp server above.

```
"HTTP and HTTPS connections to the www server."

    http            80/tcp          www www-http    # WorldWideWeb HTTP
        iptables -A FORWARD -d 23.214.219.132 -p tcp --dport http -j ACCEPT
        iptables -A FORWARD -s 23.214.219.132 -p tcp --sport http -j ACCEPT


    https           443/tcp                         # http protocol over TLS/SSL
        iptables -A FORWARD -d 23.214.219.132 -p tcp --dport https -j ACCEPT
        iptables -A FORWARD -s 23.214.219.132 -p tcp --sport https -j ACCEPT


    https           443/udp                         # http protocol over TLS/SSL
        iptables -A FORWARD -d 23.214.219.132 -p udp --dport https -j ACCEPT
        iptables -A FORWARD -s 23.214.219.132 -p udp --sport https -j ACCEPT
```

Figure 15 - Accept rules for http and https connections

Once again, the reasoning for these rules is the same as the one for the smtp server. One key aspect is that we took into consideration both tcp and udp for the https server. We did this since https can also use udp **[2]**.

```
"OpenVPN connections to the vpn-gw server."

    openvpn         1194/tcp                # OpenVPN
        iptables -A FORWARD -d 23.214.219.133 -p tcp --dport openvpn -j ACCEPT
        iptables -A FORWARD -s 23.214.219.133 -p tcp --sport openvpn -j ACCEPT



    openvpn         1194/udp                # OpenVPN
        iptables -A FORWARD -d 23.214.219.133 -p udp --dport openvpn -j ACCEPT
        iptables -A FORWARD -s 23.214.219.133 -p udp --sport openvpn -j ACCEPT
```

Figure 16 - Accept rules for OpenVPN connections

These also follow the logic used for the smtp server. We created rules for tcp and udp, since OpenVPN can be used with either protocol with minor setting changes from the user.

```
"VPN clients connected to the gateway (vpn-gw) should able to connect to the PosgreSQL service on the datastore server."

    postgres        5432/tcp        postgresql      # POSTGRES
        iptables -A FORWARD -d 192.168.10.4 -s 23.214.219.133 -p tcp --dport postgres -j ACCEPT
        iptables -A FORWARD -s 192.168.10.4 -d 23.214.219.133 -p tcp --sport postgres -j ACCEPT
```

Figure 17 - Accept rules from the vpn-gw to the datastore server

For this request to be possible, the vpn-gw must be able to forward a connection to the PostgreSQL service on the datastore server. As such, the rule with destination port postgres has the datastore server as a destination and the vpn-gw as a source.

Firewall configuration for connections to the external IP address of the firewall (using NAT)

For these connections we decided to use pre-routing, since we are connecting from the internet to an internal network. As such, it makes sense to alter the destination of the packets, since the outside shouldn't have direct access to the internal network.

Besides this, it's also necessary to allow the communications using the Forward chain, since pre-routing only changes the destination.

```
"FTP connections (in passive and active modes) to the ftp server."

    Deal with dynamic ports
        iptables -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT

    iptables -t nat -A PREROUTING -s 87.248.214.0/24 -d 87.248.214.97 -p tcp --dport ftp -j DNAT --to-destination 192.168.10.5
    iptables -A FORWARD -d 192.168.10.5 -s 87.248.214.0/24 -p tcp --dport ftp -j ACCEPT
    iptables -A FORWARD -s 192.168.10.5 -d 87.248.214.0/24 -p tcp --sport ftp-data -j ACCEPT
```

Figure 18 – Prerouting and Accept rules from the internet to the ftp server

For starters, we need to take care of the dynamic port problem in ftp. In passive mode, both ports used by the data connection are dynamic. So, we need to utilize "-m state RELATED,ESTABLISHED" in order to allow data connections for previously established ftp-control connections.

```
"SSH connections to the datastore server, but only if originated at the eden or dns2 servers."
    ssh             22/tcp                          # The Secure Shell (SSH) Protocol
    (From dns2 to datastore)
        iptables -t nat -A PREROUTING -s 87.248.214.1 -d 87.248.214.97 -p tcp --dport ssh -j DNAT --to-destination 192.168.10.4
        iptables -A FORWARD -d 192.168.10.4 -s 87.248.214.1 -p tcp --dport ssh -j ACCEPT
        iptables -A FORWARD -s 192.168.10.4 -d 87.248.214.1 -p tcp --sport ssh -j ACCEPT


    ssh             22/tcp                          # The Secure Shell (SSH) Protocol
    (From eden to datastore)
        iptables -t nat -A PREROUTING -s 87.248.214.2 -d 87.248.214.97 -p tcp --dport ssh -j DNAT --to-destination 192.168.10.4
        iptables -A FORWARD -d 192.168.10.4 -s 87.248.214.2 -p tcp --dport ssh -j ACCEPT
        iptables -A FORWARD -s 192.168.10.4 -d 87.248.214.2 -p tcp --sport ssh -j ACCEPT
```

Figure 19 – Prerouting and Accept rules from eden and dns2 to the datastore server

For this to work we put the original destination as the interface that the router uses to connect with the internet, which is then changed to the real destination.

We also put the destination port ssh on the same rules that have a destination to the datastore server, while also making sure to write the correct source.

Firewall configuration for communications from the internal network to the outside (using NAT)

As for these connections we decided to use post-routing instead, since we are connecting from the internal network to the internet. In this case, the idea is to change the source of the packets coming from the internal network, so they are in the same network as the internet (87.248.214.0/24).

Once again, it's also necessary to allow the communications using the Forward chain, since post-routing only changes the source.

```
"Domain name resolutions using DNS."
domain          53/tcp                          # name-domain server
    iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -d 87.248.214.0/24 -p tcp --dport domain -j SNAT --to-source 87.248.214.97
    iptables -A FORWARD -d 87.248.214.0/24 -s 192.168.10.0/24 -p tcp --dport domain -j ACCEPT
    iptables -A FORWARD -s 87.248.214.0/24 -d 192.168.10.0/24 -p tcp --sport domain -j ACCEPT

domain          53/udp
    iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -d 87.248.214.0/24 -p udp --dport domain -j SNAT --to-source 87.248.214.97
    iptables -A FORWARD -d 87.248.214.0/24 -s 192.168.10.0/24 -p udp --dport domain -j ACCEPT
    iptables -A FORWARD -s 87.248.214.0/24 -d 192.168.10.0/24 -p udp --sport domain -j ACCEPT
```

Figure 20 - Postrouting and Accept rules from the internal network to the internet, for the dns

In this case we define the source and destination by the networks themselves, since we aren't working with servers in particular. The source is changed to 87.248.214.97, so that the origin can be in the same network as the internet.

```
"HTTP, HTTPS and SSH connections."
    http            80/tcp          www www-http    # WorldWideWeb HTTP
        iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -d 87.248.214.0/24 -p tcp --dport http -j SNAT --to-source 87.248.214.97
        iptables -A FORWARD -d 87.248.214.0/24 -s 192.168.10.0/24 -p tcp --dport http -j ACCEPT
        iptables -A FORWARD -s 87.248.214.0/24 -d 192.168.10.0/24 -p tcp --sport http -j ACCEPT

    https           443/tcp                         # http protocol over TLS/SSL
        iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -d 87.248.214.0/24 -p tcp --dport https -j SNAT --to-source 87.248.214.97
        iptables -A FORWARD -d 87.248.214.0/24 -s 192.168.10.0/24 -p tcp --dport https -j ACCEPT
        iptables -A FORWARD -s 87.248.214.0/24 -d 192.168.10.0/24 -p tcp --sport https -j ACCEPT

    https           443/udp                         # http protocol over TLS/SSL
        iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -d 87.248.214.0/24 -p udp --dport https -j SNAT --to-source 87.248.214.97
        iptables -A FORWARD -d 87.248.214.0/24 -s 192.168.10.0/24 -p udp --dport https -j ACCEPT
        iptables -A FORWARD -s 87.248.214.0/24 -d 192.168.10.0/24 -p udp --sport https -j ACCEPT

    ssh             22/tcp                          # The Secure Shell (SSH) Protocol
        iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -d 87.248.214.0/24 -p tcp --dport ssh -j SNAT --to-source 87.248.214.97
        iptables -A FORWARD -d 87.248.214.0/24 -s 192.168.10.0/24 -p tcp --dport ssh -j ACCEPT
        iptables -A FORWARD -s 87.248.214.0/24 -d 192.168.10.0/24 -p tcp --sport ssh -j ACCEPT
```

Figure 21 - Postrouting and Accept rules from the internal network to the internet, for http, https and ssh

These use the same pattern as the one from the previous figure, with the only notable difference being the different destination and source ports.

```
"FTP connections (in passive and active modes) to external FTP servers."

    Deal with dynamic ports
        iptables -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT

    iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -d 87.248.214.0/24 -p tcp --dport ftp -j SNAT --to-source 87.248.214.97

    iptables -A FORWARD -d 87.248.214.0/24 -s 192.168.10.0/24 -p tcp --dport ftp -j ACCEPT
    iptables -A FORWARD -s 87.248.214.0/24 -d 192.168.10.0/24 -p tcp --sport ftp-data -j ACCEPT
```

Figure 22 - Postrouting and Accept rules from the internal network to the internet, for ftp connections

A big difference from the prerouting rules to this one is that the forward rules are defined using the source and destination directly (as opposed to using the source directly, but the destination from the –to-destination in prerouting). This makes sense, since in this case we know the destination and merely want to change the source.

The same ideas from the prerouting in ftp from the previous section are used here, in order to deal with the problem regarding ftp's passive mode.

Snort

IPTables rules related with snort

```
Iptables:
    http        80/tcp
    iptables -A FORWARD -d 23.214.219.132 -s 87.248.214.0/24 -p tcp --dport http -j NFQUEUE --queue-num 0
    iptables -A FORWARD -s 23.214.219.132 -d 87.248.214.0/24 -p tcp --sport http -j NFQUEUE --queue-num 0
```

Figure 23 – IPTables rules used to test snort

To test snort we need a rule that can send packets coming from the internet to a desired location, so they can enter a certain queue (NFQUEUE) and be analysed by snort. In this case, that desired location is the www server and the http port. We decided on this one, since these attacks are mainly web-based, so this seemed like the correct place to test in. This rule is placed on top of the Forward chain, to guarantee that these packets go to the queue.

SQL Injection Attacks, XSS Attacks and Snort Rules to prevent them

```
Rules for snort:
    block tcp any any -> any any (msg: "UNION SQL Injection BLOCKED"; pcre: "/((\ % 22) | (\ % 27) | (') | ("))(\s)*union/ix";
    sid:100000001; )
    block tcp any any -> any any (msg: "BOOLEAN SQL Injection BLOCKED"; content: "\" OR \"1\"=\"1\"" , nocase; sid:100000002; )
    block tcp any any -> any any (msg: "BOOLEAN SQL Injection BLOCKED"; content: "\' OR \'1\'=\'1\'" , nocase; sid:100000003; )


    block tcp any any -> any any (msg: "Cross-Site Scripting - markup injection BLOCKED"; pcre:
    "/((\%3C)|<)((\%2F)|\/)*[a-z0-9\%]+((\%3E)|>)/i"; sid:100000004; )
```

Figure 24 - Snort rules used to detect SQL Injection and Cross-Site Scripting

These rules were added to a file we named snort.rules, which is explicitly called when we execute snort (in order to load the rules).

SQL Injection Attacks – These attacks involve injecting additional SQL into a query that wasn't supposed to receive it. That way, a malicious user can force the execution of certain queries.

1) **Union Based SQL Injection** - The idea behind this attack is to use a structure like **' union**, so that a SQL command expecting a parameter, that would normally be something like [SELECT … WHERE name = 'name'], will end up looking like this: [SELECT … WHERE name = ' **' union** …]. This way the attacker can receive additional information that would normally be unavailable **[4]**. In our solution we try to block situation where the attacker uses a ' in combination with the word union, while also taking into account the possibility of encoded characters for ' and ".
2) **Boolean Based SQL Injection** – In this attack the structure of the command is changed to include a Boolean part that allows the attacker to control the end result **[4]**. This is usually [' or '1' = '1'], which will obviously return true, which the attacker can then take advantage of. This is used in identification systems, where the attacker can access an account without knowing either the username or the password. To solve this, we try to explicitly find this string, using the "content" parameter, while considering that ' or " can be used and that they can also be encoded. There are also similar attacks, that in a real situation would have to be covered as well, but it's difficult to enumerate all of them, since many similar, but different string can also return true or false.

Cross-Site Scripting – As for these attacks, they usually involve injecting tags into html that wasn't supposed to have them, so that javascript code can be executed. This code is created by attackers who want to run malicious scripts.

1) **Non-Persistent (Reflected)** - The idea behind this attack is to inject tags that revolve around forcing a script execution, but in a very immediate manner. This tags are usually added to the url, so that the malicious code is added to the original html and is then run when it arrives to the server **[5]**.
2) **Persistent** – This is a similar idea, but with a key difference: the changes in the html are more permanent. An example of this is when malicious code is added on a comment section of a website **[5]**. The comment holding malicious code is then stored in a database and every time other users load that comment they also load the malicious code, while remaining unaware of it.

   Solution - For both of these attacks we used the same rule (since their difference is more so related with persistence than with format), which simply tries to find any attempts of markup injection, by trying to match strings such as: <script>var i = 10;</script>, which are clearly trying to inject tags with scripts in them. We consider more tags than just "script", since there are more ways to force a script execution (image, alerts, …) **[5]**. So in our solution we keep things relatively non-specific, while trying not to go too far, so that we can avoid false positives. This is done by matching strings that start with "<", can optionally have a "/" and end in a ">", which is similar to what was done in **[6]**.

Tests

In terms of testing, we did everything mentioned in the file Tests.txt, which has a complete list of all the tests performed.

IPTables Testing

In general terms, we used netcat for most of the iptables testing, by using "nc –v –l port" on the server and "nc –v destinationAddress destinationPort" on the client. We did this for every rule we established previously on this report, except for the ftp related rules.

For the ftp rules, we used ftp's active and passive mode, as well as the "ls" and "get" commands, as shown below.

```
"FTP connections (in passive and active modes) to the ftp server."

    (From internet to ftp server)
        Router: tshark -i enp0s9 (Internal network)
                tshark -i enp0s10 (Internet)
        Source: ftp 87.248.214.97
            -> name: sti_extra1
            -> pass:
            -> ls
            -> get machine2.txt
            -> passive
            -> ls
            -> get machine2.txt
```

Figure 25 – FTP rules testing

For the nat related rules, we used tshark command such as the ones in the previous figure. This was done to be completely sure the source or destination addresses were being changed in the right manner (though this can also be seen in the "connection from" and "connection to" shown while using netcat's verbose option). In the case of prerouting, since the original address is on the router's interface, the fact that it reaches the correct destination also shows that the corresponding nat rule is working.

Snort Testing

To test snort, we run the following command **[snort --daq nfq --daq-var queue=0 -A alert_full -Q -c /usr/local/src/snort3/lua/snort.lua -R /etc/snort/snort.rules].** This command will run snort and take care of packets in queue 0, while using inline mode (-Q), the default configuration file (snort.lua) and our own rules file, snort.rules, which has been explained in detail before.

```
Source: nc -v 23.248.214.132 80
Destination: nc -v -l 80
    "UNION SQL Injection BLOCKED" -> send " union

    "UNION SQL Injection BLOCKED" -> send ' union


    "BOOLEAN SQL Injection BLOCKED" -> send " OR "1"="1"

    "BOOLEAN SQL Injection BLOCKED" -> send ' OR '1'='1'


    "Cross-Site Scripting - markup injection BLOCKED" -> send <script>var i = 10;</script>

    "Cross-Site Scripting - markup injection BLOCKED" -> send <b onmouseover=alert('ALERT!')>button!</b>
```

Figure 26 – Snort Rules testing

Here we use some simple examples to try to get a match from each snort rule defined in the snort section. First we do this for union-based and boolean-based SQL Injection and then for Cross-Site Scripting. In the second one (XSS) we made sure to test an example that didn't explicitly use the <script> tag, to make sure everything worked properly.

Important Annexes

In the file **Config.txt** we have the commands used to create the rules in complete detail, as well as the modifications made to the sample files.

In the **Tests.txt** file we have a list of every test performed to ensure the rules were indeed correct.

Other Annexes

Command.txt has a list of every command used to create the iptables rules, as well the ones related with snort.

The file **rules** is the saved configuration of the iptables rules.

In **snort.rules** we have all the rules used in snort, while **snort.lua** is simply the default configuration file for snort, which is completely unchanged.

References

1. https://www.linuxuprising.com/2020/07/ubuntu-how-to-free-up-port-53-used-by.html
2. https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-2000-server/cc959828(v=technet.10)?redirectedfrom=MSDN
3. https://groups.google.com/g/comp.os.linux.networking/c/wHmQARVyoNA?pli=1
4. https://www.acunetix.com/websitesecurity/sql-injection2/
5. https://owasp.org/www-community/attacks/xss/#stored-xss-attacks
6. https://topic.alibabacloud.com/a/use-snort-to-cleverly-font-colorreddetectfont-font-colorredsqlfont-font-colorredinjectionfont-and-cross-site-scripting-attacks_3_75_32817247.html